

Choosing an Appropriate Activation Function for Neural Networks

Soham Bhattacharya

February 2025

Introduction

Choosing the right activation function for a neural network is crucial for ensuring that the model trains effectively and performs well on a given task. Here are the key points to consider when selecting an activation function:

1 Non-linearity

Neural networks require **non-linear activation functions** so they can model complex relationships in the data. Without non-linearity, even a multi-layer network would behave like a single-layer network, which limits its capacity. Common non-linear activation functions include **ReLU**, **sigmoid**, and **tanh**.

2 Vanishing/Exploding Gradients

Some activation functions, such as **sigmoid** and **tanh**, suffer from **vanishing gradients** when the inputs are very large or very small, which can slow down or prevent learning, especially in deep networks. To avoid this, **ReLU** (Rectified Linear Unit) and its variants (like **Leaky ReLU** and **ELU**) are often preferred because they do not saturate for positive values of input, allowing the gradient to propagate more effectively.

3 Range of Output

- **Sigmoid**: Outputs between 0 and 1. This is often used in the output layer of binary classification models since it gives probabilities.
- **Tanh**: Outputs between -1 and 1. It's often used when the data needs to be centered around zero.

- **ReLU**: Outputs between 0 and infinity. It's widely used in hidden layers due to its simplicity and effectiveness in training deep networks.
- **Leaky ReLU** and **ELU** are used when you want to address the "dying ReLU" problem (where ReLU neurons can get stuck and stop learning).

4 Computational Efficiency

ReLU is computationally efficient because it involves simple thresholding (set negative values to zero), making it faster for training and inference compared to sigmoid or tanh, which involve exponential calculations. When building large-scale models, computational efficiency is essential for training speed and scalability.

5 Sparsity

ReLU introduces sparsity in the network because it outputs zero for any negative input. This can make the model more efficient and easier to train by allowing the network to focus on fewer, more significant activations. Sparsity can be beneficial in cases where feature selection or reduced complexity is desired.

6 Gradient Flow

For effective training, gradients should propagate well through the network. **ReLU** helps with this since it doesn't saturate for positive inputs. **Tanh** suffers from saturation for very large or very small input values, which can lead to the vanishing gradient problem. **Sigmoid** also suffers from saturation, especially for inputs far from zero, which can result in very small gradients and slow learning.

7 Task-Specific Requirements

- **Binary Classification**: For binary classification problems, **sigmoid** is often used in the output layer to produce outputs between 0 and 1, which can be interpreted as probabilities.
- **Multi-Class Classification**: For multi-class classification, **softmax** is used in the output layer to produce probability distributions over multiple classes.
- **Regression Problems**: For regression tasks, where the output is continuous, you might not use any activation function in the output layer (i.e., a **linear activation**).

8 Type of Network

- **Feedforward Networks:** **ReLU** and its variants (Leaky ReLU, ELU, etc.) are often used in hidden layers because they perform well in training deep networks.
- **Recurrent Neural Networks (RNNs):** **tanh** or **ReLU** is often used for the activation function in hidden layers. However, the issue of vanishing gradients makes **LSTMs** or **GRUs** (which use specialized gates) more suitable for long sequences.
- **Generative Networks (GANs):** **Tanh** is commonly used in the generator's output layer to ensure the output is within a certain range, such as between -1 and 1, when generating images.

9 Behavior of Activation Function

- **ReLU:** Simple, non-saturating, efficient. However, ReLU units can "die" (i.e., always output zero for all inputs) when negative inputs are encountered. **Leaky ReLU** and **ELU** can help mitigate this.
- **Sigmoid/Tanh:** Sigmoid saturates for very large positive or negative inputs, leading to vanishing gradients, and it's typically avoided for deep networks. **Tanh** also suffers from similar issues, though it is less prone to the vanishing gradient problem than the sigmoid.

10 Handling Outliers and Distribution of Inputs

- **ReLU** is robust to outliers in the positive direction but can have issues with negative inputs.
- **Tanh** helps center the data around zero but can still suffer from saturation at extreme values.
- **Softmax** ensures that outputs are normalized, which is useful in multi-class classification tasks to model probabilities across all classes.

11 Recent Trends and Improvements

- **Swish** (a smooth version of ReLU) has been proposed as a newer activation function that tends to outperform ReLU in many situations.
- **Mish**, another newer activation function, has also gained popularity as it produces even smoother and more continuous gradients, improving optimization in some cases.

12 Summary of Common Activation Functions

Activation Function	Output Range	Key Properties
ReLU	$[0, \infty)$	Fast, no saturation, simple
Leaky ReLU	$(-\infty, \infty)$	Allows small negative slope, prevents dying ReLU problem
Tanh	$[-1, 1]$	Outputs centered around 0, saturates at extreme values
Sigmoid	$(0, 1)$	Used for binary classification, saturates for extreme inputs
Softmax	$(0, 1)$ (sum=1)	Used for multi-class classification to output probabilities
Linear	$(-\infty, \infty)$	No activation, used for regression

13 Conclusion

When choosing an activation function, consider the following:

- **ReLU** is typically a good default for hidden layers because of its simplicity and efficiency.
- **Sigmoid** or **Softmax** are suitable for output layers in binary or multi-class classification tasks, respectively.
- **Tanh** can be useful for data that needs to be centered around zero, but ReLU often performs better in practice.
- For very deep networks, try **ReLU** or variants like **Leaky ReLU**, **ELU**, or **Swish**, which address vanishing gradient problems.

The right choice depends on the specific task, the architecture, and the data you're working with.